

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №4 по курсу
«Операционные системы»

Группа: М8О-214БВ-24

Студент: Кириенко А. И.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 09.12.25

Москва, 2025

Постановка задачи

Вариант 1.

1. Расчет интеграла функции $\sin(x)$ на отрезке $[A, B]$ с шагом e :

Сигнатура функции: `float sin_integral(float a, float b, float e);`

- Реализация №1: Подсчет интеграла методом прямоугольников;
- Реализация №2: Подсчет интеграла методом трапеций.

2. Расчет производной функции $\cos(x)$ в точке a с приращением dx :

Сигнатура функции: `float cos_derivative(float a, float dx);`

- Реализация №1: $f'(x) = (f(a + dx) - f(a)) / dx$
- Реализация №2: $f'(x) = (f(a + dx) - f(a - dx)) / (2dx)$

Общий метод и алгоритм решения

Использованные системные вызовы:

- `ssize_t read(int fd, void *buf, size_t count)` - чтение из стандартного ввода (POSIX)
- `ssize_t write(int fd, const void *buf, size_t count)` - запись в стандартный вывод (POSIX)
- `void *dlopen(const char *filename, int flags)` - загрузка динамической библиотеки
- `void *dlsym(void *handle, const char *symbol)` - получение адреса функции из библиотеки
- `int dlclose(void *handle)` - закрытие библиотеки
- `char *dlerror(void)` - получение ошибки

1. Созданы две динамические библиотеки с разными реализациями математических функций:

lib1.c (первая реализация)

lib2.c (вторая реализация)

2. Создание заголовочного файла

contract.h определяет «контракт» - что должна реализовать библиотека

3. Компиляция библиотек:

- Компиляция с -fPIC → Объектные файлы
- Линковка с -shared → Динамические библиотеки

Компиляция в объектный файл с позиционно-независимым кодом

```
gcc -fPIC -c lib1.c -o lib1.o
```

Создание динамической библиотеки

```
gcc -shared -o lib1.so lib1.o -lm
```

-shared = создать shared library (динамическую)

-lm = подключить математическую библиотеку

Для lib2.so (аналогично):

```
gcc -fPIC -c lib2.c -o lib2.o
```

```
gcc -shared -o lib2.so lib2.o -lm
```

4. Разработаны две тестовые программы:

program1.c

- использует динамическую библиотеку lib1.so на этапе компиляции.
- связывание происходит с помощью параметров линковщика -L. -lmylib
- не поддерживается подключение между библиотеками во время выполнения

Алгоритм работы program1.c

1. Вывести меню команд

2. Читать команды в цикле:

- Если '1': вызвать sin_integral

- Если '2': вызвать cos_derivative

- Если 'q': выйти

3. Для каждой команды:

- Распарсить аргументы
- Вызвать соответствующую функцию
- Вывести результат

program2.c

- загружает библиотеки во время выполнения
- использует системные вызовы dlopen(), dlsym(), dlclose()
- поддерживает переключение между lib1.so и lib2.so

Алгоритм работы program2.c

1. Загрузить начальную библиотеку (lib1.so)

2. Получить указатели на функции

3. Вывести меню команд

4. Читать команды в цикле:

- Если '0': переключить библиотеку
- Если '1': вызвать sin_integral через указатель
- Если '2': вызвать cos_derivative через указатель
- Если 'q': выйти

5. Компиляция программ

Создаем символическую ссылку

```
ln -sf lib1.so libmylib.so
```

libmylib.so → указывает на lib1.so

Компилируем программу

```
gcc -c program1.c -o program1.o
```

Линкуем с библиотекой

```
gcc -o program1 program1.o -L. -lmylib -lm
```

-L. = искать библиотеки в текущей директории

-lmylib = использовать библиотеку libmylib.so

-lm = математическая библиотека

```
# Компилируем программу2

gcc -c program2.c -o program2.o

# Линкуем с библиотекой динамической загрузки

gcc -o program2 program2.o -ldl -lm

# -ldl = библиотека для dlopen, dlsym, dlclose

# -lm = математическая библиотека
```

Код программы

lib1.c

```
#include <math.h>

float sin_integral(float a, float b, float e) {
    float integral = 0.0;
    float x = a;
    while (x < b) {
        float next_x = x + e;
        if (next_x > b) next_x = b;
        float mid = (x + next_x) / 2;
        integral += sin(mid) * e;
        x = next_x;
    }
    return integral;
}

float cos_derivative(float a, float dx) {
    return (cos(a + dx) - cos(a)) / dx;
}
```

lib2.c

```
#include <math.h>

float sin_integral(float a, float b, float e) {
    float integral = 0.0;
    float x = a;
    while (x < b) {
        float next_x = x + e;
        if (next_x > b) next_x = b;
        float step = next_x - x;
        integral += (sin(x) + sin(next_x)) * step / 2;
        x = next_x;
    }
    return integral;
}

float cos_derivative(float a, float dx) {
    return (cos(a + dx) - cos(a - dx)) / (2 * dx);
}
```

contract.h

```
#ifndef CONTRACT_H
#define CONTRACT_H

float sin_integral(float a, float b, float e);
float cos_derivative(float a, float dx);
#endif
```

program1.c

```
#include "contract.h"

#include <unistd.h>

#include <string.h>

#define BUFFER_SIZE 1024

void write_string(const char* str) {
    write(STDOUT_FILENO, str, strlen(str));
}

void write_int(int num) {
    char buffer[32];

    int i = 0;

    int is_negative = 0;

    if (num < 0) {
        is_negative = 1;
        num = -num;
    }

    if (num == 0) {
        buffer[i++] = '0';
    } else {
        char temp[32];

        int j = 0;

        while (num > 0) {
            temp[j++] = '0' + (num % 10);

            num /= 10;
        }
    }
}
```

```

    }

    while (j > 0) {

        buffer[i++] = temp[--j];

    }

}

if (is_negative) {

    char temp[32];

    temp[0] = '-';

    for (int k = 0; k < i; k++) {

        temp[k + 1] = buffer[k];

    }

    write(STDOUT_FILENO, temp, i + 1);

} else {

    write(STDOUT_FILENO, buffer, i);

}

}

void write_float(float num) {

    if (num < 0) {

        write_string("-");

        num = -num;

    }

    int int_part = (int)num;

    write_int(int_part);

    write_string(".");

    float frac = num - int_part;

```



```

for (int i = 0; i < 6; i++) {

    frac *= 10;

    int digit = (int)frac;

    char c = '0' + digit;

    write(STDOUT_FILENO, &c, 1);

    frac -= digit;

}

write_string("\n");
}

float parse_float(const char* str) {

    float result = 0.0;

    float fraction = 0.1;

    int sign = 1;

    int decimal = 0;

    if (*str == '-') {

        sign = -1;

        str++;

    }

    while (*str) {

        if (*str == '.') {

            decimal = 1;

        } else if (*str >= '0' && *str <= '9') {

            if (decimal) {

                result += (*str - '0') * fraction;

                fraction *= 0.1;

            } else {

```

```

        result = result * 10 + (*str - '0');

    }

}

str++;

}

return result * sign;
}

int main() {

    char buffer[BUFFER_SIZE];

    char command[10];

    float args[3];

    write_string("Program 1 (compile-time linking)\n");

    write_string("Commands:\n");

    write_string(" 0 - not used in program 1\n");

    write_string(" 1 a b e - calculate sin integral from a to b with step e\n");

    write_string(" 2 a dx - calculate cos derivative at point a with dx\n");

    write_string(" q - quit\n");

    while (1) {

        write_string("> ");

        int n = read(STDIN_FILENO, buffer, BUFFER_SIZE - 1);

        if (n <= 0) break;

        buffer[n] = '\0';

```

```
if (buffer[0] == 'q' || buffer[0] == 'Q') {  
    break;  
}  
  
int num_args = 0;  
  
char* token = strtok(buffer, " \t\n");  
  
if (token == NULL) continue;  
  
strcpy(command, token);  
  
if (strcmp(command, "1") == 0) {  
    token = strtok(NULL, " \t\n");  
    while (token != NULL && num_args < 3) {  
        args[num_args++] = parse_float(token);  
        token = strtok(NULL, " \t\n");  
    }  
  
    if (num_args == 3) {  
        float result = sin_integral(args[0], args[1], args[2]);  
        write_string("Result: ");  
        write_float(result);  
    } else {  
        write_string("Error: need 3 arguments: a b e\n");  
    }  
}  
  
else if (strcmp(command, "2") == 0) {  
    token = strtok(NULL, " \t\n");  
    while (token != NULL && num_args < 2) {
```

```

        args[num_args++] = parse_float(token);

        token = strtok(NULL, " \t\n");
    }

    if (num_args == 2) {

        float result = cos_derivative(args[0], args[1]);

        write_string("Result: ");

        write_float(result);

    } else {

        write_string("Error: need 2 arguments: a dx\n");

    }

}

else if (strcmp(command, "0") == 0) {

    write_string("Command '0' not used in program 1 (compile-time
linking)\n");

}

else {

    write_string("Unknown command. Use 1, 2, or q\n");

}

}

return 0;
}

```

program2.c

```

#include <unistd.h>

#include <string.h>

#include <dlfcn.h>

#include <stdlib.h>

```

```
#define BUFFER_SIZE 1024

void write_string(const char* str) {
    write(STDOUT_FILENO, str, strlen(str));
}

void write_int(int num) {
    char buffer[32];

    int i = 0;

    int is_negative = 0;

    if (num < 0) {
        is_negative = 1;
        num = -num;
    }

    if (num == 0) {
        buffer[i++] = '0';
    } else {
        char temp[32];

        int j = 0;

        while (num > 0) {
            temp[j++] = '0' + (num % 10);

            num /= 10;
        }

        while (j > 0) {
            buffer[i++] = temp[--j];
        }
    }
}
```

```

    }

    if (is_negative) {
        char temp[32];

        temp[0] = '-';

        for (int k = 0; k < i; k++) {
            temp[k + 1] = buffer[k];
        }

        write(STDOUT_FILENO, temp, i + 1);
    } else {
        write(STDOUT_FILENO, buffer, i);
    }
}

```

```

void write_float(float num) {
    if (num < 0) {
        write_string("-");
        num = -num;
    }

    int int_part = (int)num;
    write_int(int_part);
    write_string(".");

    float frac = num - int_part;

    for (int i = 0; i < 6; i++) {
        frac *= 10;

        int digit = (int)frac;

        char c = '0' + digit;
    }
}

```

```

        write(STDOUT_FILENO, &c, 1);

        frac -= digit;
    }

    write_string("\n");
}

float parse_float(const char* str) {
    float result = 0.0;

    float fraction = 0.1;

    int sign = 1;

    int decimal = 0;

    if (*str == '-') {
        sign = -1;
        str++;
    }

    while (*str) {
        if (*str == '.') {
            decimal = 1;
        } else if (*str >= '0' && *str <= '9') {
            if (decimal) {
                result += (*str - '0') * fraction;
                fraction *= 0.1;
            } else {
                result = result * 10 + (*str - '0');
            }
        }

        str++;
    }

```

```

    }

    return result * sign;
}

int main() {

    char buffer[BUFFER_SIZE];

    char command[10];

    float args[3];

    void* library = NULL;

    float (*sin_integral)(float, float, float) = NULL;

    float (*cos_derivative)(float, float) = NULL;

    int current_lib = 1;

    write_string("Program 2 (runtime loading)\n");

    write_string("Commands:\n");

    write_string(" 0 - switch between lib1.so and lib2.so\n");

    write_string(" 1 a b e - calculate sin integral from a to b with step e\n");

    write_string(" 2 a dx - calculate cos derivative at point a with dx\n");

    write_string(" q - quit\n");

    library = dlopen("./lib1.so", RTLD_LAZY);

    if (!library) {

        write_string("Error loading lib1.so\n");

        return 1;

    }

```



```
sin_integral = dlsym(library, "sin_integral");
cos_derivative = dlsym(library, "cos_derivative");

if (!sin_integral || !cos_derivative) {
    write_string("Error getting functions\n");
    dlclose(library);
    return 1;
}

write_string("Loaded lib1.so (rectangles, first derivative formula)\n");

while (1) {
    write_string("> ");

    int n = read(STDIN_FILENO, buffer, BUFFER_SIZE - 1);
    if (n <= 0) break;

    buffer[n] = '\0';

    if (buffer[0] == 'q' || buffer[0] == 'Q') {
        break;
    }

    int num_args = 0;
    char* token = strtok(buffer, " \t\n");

    if (token == NULL) continue;

    strcpy(command, token);
```

```
if (strcmp(command, "0") == 0) {  
    dlclose(library);  
  
    if (current_lib == 1) {  
        library = dlopen("./lib2.so", RTLD_LAZY);  
        current_lib = 2;  
        write_string("Switched to lib2.so (trapezoids, second derivative  
formula)\n");  
    } else {  
        library = dlopen("./lib1.so", RTLD_LAZY);  
        current_lib = 1;  
        write_string("Switched to lib1.so (rectangles, first derivative  
formula)\n");  
    }  
  
    if (!library) {  
        write_string("Error loading library\n");  
        return 1;  
    }  
  
    sin_integral = dlsym(library, "sin_integral");  
    cos_derivative = dlsym(library, "cos_derivative");  
  
    if (!sin_integral || !cos_derivative) {  
        write_string("Error getting functions\n");  
        dlclose(library);  
        return 1;  
    }  
}
```

```
else if (strcmp(command, "1") == 0) {

    token = strtok(NULL, " \t\n");

    while (token != NULL && num_args < 3) {

        args[num_args++] = parse_float(token);

        token = strtok(NULL, " \t\n");

    }

    if (num_args == 3) {

        float result = sin_integral(args[0], args[1], args[2]);

        write_string("Result: ");

        write_float(result);

    } else {

        write_string("Error: need 3 arguments: a b e\n");

    }

}

else if (strcmp(command, "2") == 0) {

    token = strtok(NULL, " \t\n");

    while (token != NULL && num_args < 2) {

        args[num_args++] = parse_float(token);

        token = strtok(NULL, " \t\n");

    }

    if (num_args == 2) {

        float result = cos_derivative(args[0], args[1]);

        write_string("Result: ");

        write_float(result);

    } else {

        write_string("Error: need 2 arguments: a dx\n");

    }

}
```

```

    }

    else {

        write_string("Unknown command. Use 0, 1, 2, or q\n");

    }

}

if (library) {

    dlclose(library);

}

return 0;

}

```

Протокол работы программы

```

> q
anastasia@DESKTOP-5B216BB:/mnt/c/Users/kirie/.vscode/OS_laba4$ ./program1
Program 1 (compile-time linking)
Commands:
  0 - not used in program 1
  1 a b e - calculate sin integral from a to b with step e
  2 a dx - calculate cos derivative at point a with dx
  q - quit
> 1 0 3.14159 0.001
Result: 2.000006
> 2 1.5708 0.0001
Result: -1.000166
> q
anastasia@DESKTOP-5B216BB:/mnt/c/Users/kirie/.vscode/OS_laba4$ ./program2
Program 2 (runtime loading)
Commands:
  0 - switch between lib1.so and lib2.so
  1 a b e - calculate sin integral from a to b with step e
  2 a dx - calculate cos derivative at point a with dx
  q - quit
Loaded lib1.so (rectangles, first derivative formula)
> 1 0 3.14159 0.001
Result: 2.000006
> 0
Switched to lib2.so (trapezoids, second derivative formula)
> 1 0 3.14159 0.001
Result: 2.000000
> 2 1.5708 0.0001
Result: -1.000166
> 0
Switched to lib1.so (rectangles, first derivative formula)
> 2 1.5708 0.0001
Result: -1.000166
> q
anastasia@DESKTOP-5B216BB:/mnt/c/Users/kirie/.vscode/OS_laba4$

```

Вывод

В результате выполнения лабораторной работы были успешно освоены методы работы с динамическими библиотеками в операционной системе Linux. Были созданы две библиотеки с разными реализациями математических функций и разработаны программы, демонстрирующие принципиально разные подходы к их использованию: статическое связывание на этапе компиляции и динамическая загрузка во время выполнения. Полученные знания позволяют выбирать оптимальный подход в зависимости от требований конкретного проекта к расширяемости, производительности и простоте поддержки.